ALGORITHM 5: Source-DPOR algorithm for general blocking.

Initial call: $Explore(\langle\rangle, \emptyset)$

```
1  Explore(E, Sleep)
2      foreach p that is blocked in s_[E] do
3          foreach e ∈ dom(E) do
4              let E' = pre(E, e);
5              let w = notdep(e, E);
6              foreach subsequence u of w such that u ⊑_[E'] w do
7                  if ê disables p after E.u then
8                      let v = u.p;
9                      if I_[E'](v) ∩ backtrack(E') = ∅ then
10                         add some q' ∈ I_[E'](v) to backtrack(E');
11     if ∃p ∈ (enabled(s_[E]) \ Sleep) then
12         backtrack(E) := {p};
13         while ∃p ∈ (backtrack(E) \ Sleep) do
14             foreach e ∈ dom(E) such that(e ≺̃_E.p next_[E](p)) do
15                 let E' = pre(E, e);
16                 let v = notdep(e, E).p;
17                 if I_[E'](v) ∩ backtrack(E') = ∅ then
18                     add some q' ∈ I_[E'](v) to backtrack(E');
19             let Sleep' = {q ∈ Sleep | E ⊢ p◇q};
20             Explore(E.p, Sleep');
21             add p to Sleep;
```

Let us from this example extract a general pattern for how to extend the race detection phase of our algorithms. In this race detection, we should try to find a sequence that plays the role of v, as defined on line 6 in Algorithm 3 and on line 12 of Algorithm 4. In Algorithms 3 and 4, we could take v to be the sequence of all statements that do not happen-after p, i.e., $q.r.s$, since any happens-before closed subset of v will enable s. However, in the present case, it may happen that only a specific subsequence of v can enable s. In this case, the subsequence is r, implying that we should take v as $r.s$. Continuing with the example in Fig. 9, we see that there are several possible choices for the sequence v, but of all these only r will lead to a state where s can be disabled by another process. It is, of course, possible to develop optimization schemes that would disregard many of these choices, but we here refrain from doing this, in order to keep the structure of the algorithm simple.

Let us now incorporate these insights into general extensions of Algorithms 3 and 4. The generalization of Algorithm 3 is shown in Algorithm 5. The extended race detection phase is shown at lines 2–10. Whenever a process p is blocked, we try to detect whether some other process $\widehat{e}$ disables p. As described above, we may have to move $\widehat{e}$ past some subsequence u in order to make the disabling visible. As in Algorithm 3, the algorithm first extracts the sequence w that does not happen-after e. Thereafter, it considers all possible subsequences u of w that are consistent w.r.t. the happens-before relation. Whenever $\widehat{e}$ disables some process p after u, then the sequence $u.p$ is used as v in the race detection phase. The rest of the race detection proceeds as before.